AI와 함께 일하는 기본기

Claude Code를 배우기 전에, 그리고 배우면서 이해해야 할 것

명령어보다
맥락을 먼저.

AI가 무엇을 보고, 무엇을 바꾸며,
나는 무엇을 확인해야 하는지 배우는 교재

01 **이해** AI와 도구의 차이 → 02 **준비** 파일과 작업 공간 → 03 **협력** 설명과 피드백 → 04 **검증** 안전하게 확인

기초 중심 재구성판 · 개인 학습용

인프런 짐코딩 강좌 + Anthropic 공식 교육 대조
쉬운 설명 · 구조도 · 직접 해보기 · 복습 질문

자료 확인일 2026.10.21 · A4 인쇄용
강의의 공식 교재나 공식 한국어 번역본이 아닙니다.

이 책에서 바꾼 것

많이 아는 것보다, 내 작업을 설명하고 확인할 수 있는 것을 목표로 합니다.

구분	뜻
원래 자료	140강 압축·7개 파트. 설치와 여러 도구, 대형 프로젝트를 넓게 다룹니다.
이번 재구성	초보자의 이해에 필요한 순서로 다시 설명합니다. 기초는 본문에, 고급 도구는 선택 학습으로 옮겼습니다.
인프런 최신 확인	공개 목차 30개 섹션·191개 항목, 수강 화면 167개 수업·24개 미션. 두 숫자는 서로 다른 표시 범위입니다.
공식 교육 추가	AI Fluency의 4D, AI의 한계, 설명·판단의 반복, 책임 있는 사용을 반영했습니다.

> **확인한 범위와 확인하지 않은 범위**
> 강사 기초 교안 12개와 최신 수업 159·160·162·165·167의 첨부 자료를 읽었습니다. 전체 167개 영상과 모든 자막을 시청·검증한 것은 아닙니다. 공개 최신 목차 전체를 연결표에 반영하되, 모든 강의 본문을 새로 완역한 책은 아닙니다.

원본은 그대로 보존합니다

열어 두신 원본 파일은 덮어쓰지 않았습니다. 이 책은 원본의 실전 개발 세부 절차를 대체하는 전집이 아니라, 그것을 이해할 수 있도록 순서를 바꾼 기초 교재입니다. 원본의 제목·수치를 무조건 최신 사실로 승격하지 않았습니다.

공식 내용과 편집 설명을 구분합니다

4D와 제품 기능은 출처를 표시했습니다. 비유, 예제, 실습, 학습 순서는 이해를 돕기 위해 새로 만들었습니다. 특정 도구 설치, 유료 구독, 실제 서비스 배포를 대신 실행하지 않았습니다.

한 번에 하나씩 배우는 길

이 순서는 공식 강좌 순서가 아니라, 처음 배우는 사람을 위한 편집 순서입니다.

01 **AI를 이해하기**

4-7쪽 · 모델과 도구, 한계, 4D, 일 나누기

02 **내 작업 공간 준비하기**

8-13쪽 · 파일, 터미널, 설치, 첫 대화, 권한

03 **잘 설명하고 판단하기**

14-17쪽 · 요청, 컨텍스트, 메모리, 피드백

04 **작게 만들고 확인하기**

18-23쪽 · 저장, 작업 순환, 웹 구조, 오류, 안전

05 **나 혼자 한 바퀴 완주하기**

24-31쪽 · 작은 프로젝트, 검증, 기록지, 자가 평가

06 **필요할 때만 확장하기**

32-40쪽 · 심화 지도, 정정 사항, 강좌 연결, 용어, 출처

> **추천 학습법: 하루 20~30분**
> ① 두 쪽 읽기 → ② 모르는 말을 내 말로 바꾸기 → ③ 실습 하나 → ④ 책을 덮고 설명하기. 시간은 편집상 제안이며, 빨리 끝내야 하는 기준이 아닙니다.

다음으로 넘어가는 기준

단축키를 외웠는지가 아닙니다. “이것이 무엇인지, 왜 필요한지, 잘못되면 무엇을 확인할지”를 자기 말로 설명할 수 있으면 넘어가세요. 막히면 앞 단원으로 돌아가도 됩니다.

04. 모델과 도구는 다릅니다

AI 모델은 답을 만드는 부분이고, 도구는 실제 일을 할 수 있게 해 주는 부분입니다.

사람에 비유하면

글을 잘 쓰는 친구가 있다고 생각해 보세요. 친구가 편지를 써 주는 것과 내 컴퓨터의 파일을 직접 고치는 것은 다릅니다. 파일을 고치려면 그 파일을 볼 방법과 바꿀 권한이 따로 필요합니다. AI도 비슷합니다. 단, AI가 사람처럼 이해하거나 책임을 진다는 뜻은 아닙니다.

01
사람의 목표
무엇을 할지 정함

→

02
모델
정보를 바탕으로 답을 생성

→

03
도구
파일 읽기·수정·실행

→

04
관찰 결과
실제로 바뀐 것을 확인

구분	뜻
Claude 모델	글·코드 등을 생성하는 모델. 모델 이름과 버전은 바뀔 수 있습니다.
Claude Code	모델에 파일·명령 실행 등의 도구와 작업 맥락을 제공하는 제품입니다.
터미널·편집기	명령을 입력하거나 파일을 보는 화면입니다. 모델 자체가 아닙니다.
에이전트	목표에 맞춰 도구 사용과 결과 확인을 반복하는 작업 방식입니다. 완전한 자율·정확성을 뜻하지 않습니다.

> **가장 중요한 구분**
> “AI가 고쳤다고 말함”과 “파일이 실제로 바뀌고 원하는 대로 작동함”은 다릅니다. 마지막은 반드시 결과로 확인해야 합니다.

책을 덮고 말해 보세요

모델, Claude Code, 터미널을 각각 한 문장으로 설명해 보세요.

05. 유창한 답이 정답은 아닙니다

생성형 AI는 배운 패턴과 지금 주어진 정보를 이용해 새로운 답을 만듭니다.

학습할 때

많은 자료의 패턴을 배우고, 이후 추가 학습으로 지시를 따르는 방식 등을 익힙니다. 학습한 모든 문서를 그대로 꺼내 보여 주는 검색창은 아닙니다.

대화할 때

현재 대화와 읽어 온 자료를 바탕으로 응답을 생성합니다. 대화에 정보를 넣는 것은 보통 모델 자체를 다시 훈련하는 것과 다릅니다.

구분	뜻
그럴듯한 오류	없는 사실·출처·기능을 사실처럼 말할 수 있습니다. 말투의 자신감은 정확도의 증거가 아닙니다.
지식의 시차	모델이 최신 내용을 저절로 아는 것은 아닙니다. 검색이나 문서 읽기를 실제로 했는지 확인해야 합니다.
한 번에 보는 범위	현재 참고할 수 있는 정보량에는 한계가 있습니다. 이전 대화가 모두 계속 보인다고 가정하지 않습니다.
추론·계산의 실수	논리나 계산도 틀릴 수 있습니다. 중요한 계산은 계산기·코드·원자료와 대조합니다.

> **작은 실험**
>
> 가상의 행사 정보를 두 문장으로 만든 뒤 AI에게 요약을 요청하세요. 일부러 장소를 비워 두세요. AI가 장소를 만들어 내는지, 빠진 정보라고 표시하는지 관찰하세요. 정답은 “모르면 묻거나 미정으로 남기는 것”입니다.

책을 덮고 말해 보세요

AI가 최신 요금표를 말했습니다. “출처가 있다” 외에 무엇을 확인해야 할까요?

06. 잘 쓰는 능력은 네 가지입니다

Anthropic의 AI Fluency는 좋은 질문만이 아니라, 협력 전체를 다룹니다.

Delegation

일 나누기

내가 할 일과 AI에게 맡길 일을 정합니다.

Description

설명하기

결과·방법·대화 방식을 알려 줍니다.

Discernment

살펴보고 판단하기

맞는지, 쓸모 있는지, 과정이 적절한지 봅니다.

Diligence

책임 있게 사용하기

정보를 보호하고, 공유할 결과를 검증합니다.

행사 안내문을 만든다면

나는 날짜·장소·대상·정확한 요금을 확인합니다. AI는 초안을 쓰거나 설명을 쉽게 바꿀 수 있습니다. 나는 초안을 원본과 대조하고, 필요한 경우 AI의 기여를 밝힌 뒤 안내문을 내보냅니다.

> **4D는 한 번만 지나가는 계단이 아닙니다**
> 네 역량은 작업 내내 함께 필요합니다. 결과를 살펴보다가 일의 범위를 다시 나눌 수도 있고, 설명을 고칠 수도 있습니다. 마지막 책임은 “AI가 그렇게 말했다”로 없어지지 않습니다.

세 가지 활용 방식

정해진 일을 시키는 자동화, 함께 생각하는 보조·협업, 목표를 주고 독립적으로 움직이게 하는 대행 방식이 있습니다. 이 책에서는 먼저 함께 보고 확인하는 협업부터 익힙니다.

책을 덮고 말해 보세요

“AI에게 전부 맡기면 AI를 잘 쓰는 것이다.” 이 말에서 빠진 것은 무엇인가요?

07. 맡기기 전에 일을 나눕니다

작업을 쪼개면, AI가 틀려도 어디를 다시 봐야 할지 알 수 있습니다.

01 **목표 정하기** 어떤 결과가 필요할까 → 02 **작게 나누기** 한 번에 확인할 크기 → 03 **담당 정하기** 사람·AI·함께 → 04 **완료 기준** 무엇을 보고 끝낼까

작업	누가 할까	확인할 것
안내문 초안	AI에게 맡길 수 있음	사람이 제공한 날짜·장소만 사용
대상 독자 결정	사람이 결정	실제 이용자와 상황을 아는가
요금·계약 조건	사람이 원자료 확인	최신 원본과 일치하는가
쉬운 말로 바꾸기	AI와 함께	뜻을 바꾸거나 조건을 빼지 않았는가
외부 발송	최종 확인 후 사람 승인	받는 사람·내용·첨부가 정확한가

처음에는 되돌리기 쉬운 작업부터

개인 연습 문장, 복사해 둔 파일, 가짜 데이터가 좋습니다. 실제 결제, 고객 정보, 계약, 중요한 삭제는 연습 재료로 쓰지 않습니다. “할 수 있는 일”과 “해도 되는 일”은 다릅니다.

지금 적어 보세요

내가 만들 결과 한 가지: _____

AI에게 맡길 작은 일: _____

내가 직접 결정할 것: _____

완료를 확인할 증거: _____

책을 덮고 말해 보세요

“홈페이지를 만들어 줘”를 첫 20분에 확인할 수 있는 작은 일로 바꿔 보세요.

08. 파일과 폴더는 작업의 주소입니다

AI가 엉뚱한 파일을 고치지 않게 하려면, 먼저 내가 위치를 알아야 합니다.

공책과 서랍으로 생각하기

파일은 내용을 담은 공책, 폴더는 공책을 모아 놓은 서랍입니다. 경로는 그 공책이 어디에 있는지 알려 주는 주소입니다. 프로젝트는 한 작업에 필요한 파일을 모은 폴더라고 생각해도 좋습니다.

ai-practice / 연습 전용 폴더	
README.md	목표와 실행 방법을 적은 안내문
index.html	브라우저에서 열어 볼 연습 결과
CLAUDE.md	AI에게 반복해서 알려 줄 프로젝트 규칙

구분	뜻
파일 이름	index.html에서 .html은 파일의 종류를 나타내는 확장자입니다. 이름이 비슷해도 다른 파일일 수 있습니다.
현재 폴더	터미널이 지금 바라보는 위치입니다. 같은 명령도 위치가 다르면 다른 파일에 영향을 줍니다.
상대 경로	현재 위치를 기준으로 찾습니다. ./는 현재 폴더, ../는 한 단계 위를 뜻합니다.
숨김 파일	점으로 시작하는 파일·폴더 등은 화면에서 숨겨질 수 있습니다. .env는 비밀 설정을 담을 수 있어 특히 주의합니다.

> **연습 공간 원칙**
> 문서·바탕화면 전체를 작업 공간으로 쓰지 마세요. 이름을 알아보기 쉬운 별도 연습 폴더를 만들고, 민감정보와 업무 원본을 넣지 않습니다.

책을 덮고 말해 보세요

파일 이름은 같은데 다른 내용이 열렸다면, 무엇을 먼저 확인할까요?

09. 터미널은 글자로 조작하는 창입니다

검은 화면을 외우는 공부가 아니라, 내 위치와 행동을 확인하는 공부입니다.

터미널과 셸

터미널은 글자로 명령을 주고받는 창입니다. 셸은 그 명령을 해석하는 프로그램입니다. macOS에서는 보통 zsh, Windows에서는 PowerShell이나 CMD를 만납니다. 창이 비슷해 보여도 명령이 같지 않을 수 있습니다.

할 일	macOS 셸	PowerShell	CMD
현재 위치 보기	pwd	Get-Location 또는 pwd	cd
파일 목록 보기	ls	Get-ChildItem 또는 ls	dir
폴더 이동	cd "폴더경로"	cd "폴더경로"	cd "폴더경로"
상위 폴더 이동	cd ..	cd ..	cd ..
실행 중지	보통 Ctrl+C	보통 Ctrl+C	보통 Ctrl+C

> **지금은 세 가지만**
> ① 현재 위치를 봅니다. ② 파일 목록을 봅니다. ③ 연습 폴더로 이동합니다. 삭제·권한 변경 명령은 이 단원의 연습 대상이 아닙니다.

macOS에서 순서대로 입력하는 예시 · 경로는 실제 값으로 바꿉니다

```
pwd
ls
cd "내 연습 폴더의 실제 경로"
pwd
```

복사하기 전에

명령 한 줄이 무슨 일을 하는지 설명할 수 있어야 합니다. 따옴표 안의 경로도 확인하세요. 인터넷 명령을 통째로 붙여넣고 관리자 권한을 주는 것은 위험할 수 있습니다.

책을 덮고 말해 보세요

명령어 오류가 났을 때 운영체제뿐 아니라 함께 알려 줘야 할 프로그램 이름은 무엇인가요?

10. 설치는 필요한 것만 합니다

편집기·플러그인을 많이 설치한다고 기본기가 생기는 것은 아닙니다.

구분	뜻
AI 대화 연습	접근 가능한 AI 대화 도구. 공식 AI Fluency 활동은 유료 구독 없이도 가능합니다.
Claude Code 연습	Claude Code를 사용할 수 있는 구독·Console 또는 지원 제공자 접근이 필요합니다. 접속·과금 경로를 먼저 확인합니다.
편집기	텍스트를 열고 고칠 수 있으면 시작할 수 있습니다. 특정 유료 편집기는 필수가 아닙니다.
Node.js·npm	JavaScript 프로젝트에서 필요할 수 있습니다. 현재 공식 네이티브 설치 명령의 선행 필수품으로 혼동하지 않습니다.

macOS · 공식 네이티브 설치

```
curl -fsSL https://claude.ai/install.sh
| bash
```

Windows PowerShell · 공식 네이티브 설치

```
irm https://claude.ai/install.ps1 | iex
```

> **이 명령은 내려받은 설치 코드를 실행합니다**
>
> 공식 주소와 운영체제를 확인한 뒤 본인이 실행하세요. 화면에 맞지 않으면 멈추고 공식 설치 문서를 확인합니다. 이 책 제작 과정에서는 사용자 컴퓨터에 설치를 실행하지 않았습니다. 필요하면 신뢰하는 성인·관리자의 도움을 받으세요.

새 터미널에서 설치 확인 · 버전 번호가 나오는지 봅니다

```
claude --version
```

기본기 학습은 4-9쪽과 14-17쪽부터 진행해도 됩니다. 설치·계정 문제 때문에 AI 이해 공부 전체를 멈출 필요는 없습니다.

11. 설치 확인과 문제 찾기

명령을 반복하기보다, 어디에서 막혔는지를 구분합니다.

01 **명령 찾기** claude --version → 02 **프로그램 실행** 연습 폴더에서 claude → 03 **로그인** 화면의 공식 절차 → 04 **접근 확인** 모델 응답·요금 경로

구분	뜻
command not found / 인식 불가	명령을 찾지 못함. 새 터미널을 열어 확인하고 공식 PATH 안내를 봅니다. 설치 실패라고 곧바로 단정하지 않습니다.
irm을 모른다고 함	CMD에 PowerShell 명령을 넣은 것인지 확인합니다. 셸 이름부터 확인하세요.
403·HTML 관련 오류	설치 주소 접근·네트워크 등 문제일 수 있습니다. 오류 문구를 보존하고 공식 문제 해결 문서와 대조합니다.
로그인·권한 오류	계정·선택한 인증 경로·사용 가능 플랜을 확인합니다. 설치 성공과 서비스 접근 성공은 다릅니다.
응답이 느리거나 중단	네트워크·서비스 상태·사용량 제한 등을 구분합니다. 무조건 재설치하지 않습니다.

PATH를 쉬운 말로

터미널이 프로그램을 찾는 주소 목록입니다. 프로그램이 있어도 그 위치가 목록에 없으면 이름만으로 실행하지 못할 수 있습니다. 설정 파일을 아무렇게나 고치기보다 공식 문제 해결 절차를 따르세요.

> **기록할 네 가지**
> 운영체제 / 셸 이름 / 내가 넣은 정확한 명령 / 나온 오류 문구. 단, 로그인 코드·비밀번호·API 키는 가리고 공유합니다.

책을 덮고 말해 보세요

“버전 번호가 나왔다”로 확인된 것과, 아직 확인되지 않은 것을 나눠 보세요.

12. 첫 대화는 읽기부터 시작합니다

처음부터 앱 전체를 만들지 않고, AI가 무엇을 보고 있는지 확인합니다.

- [ ] 연습 폴더에 민감정보가 없는지 확인했습니다.
- [ ] 현재 경로를 확인했습니다.
- [ ] 직접 만든 짧은 README.md 하나만 넣었습니다.

README.md에 직접 적을 내용

```
# 나의 연습
목표: 오늘 할 일 세 가지를 보여 주는 간단한 웹 문서
범위: 내 컴퓨터에서만 열기
금지: 외부 서비스 추가 업로드, 패키지 설치, 배포
```

Claude Code 대화창에 입력할 요청

```
현재 폴더의 README.md만 읽고 다음을 알려 주세요.
1. 이 연습의 목표와 범위
2. 아직 정해지지 않은 것
3. 다음에 할 가장 작은 한 가지
지금은 파일을 수정하거나 명령을 실행하지 마세요.
```

답을 읽을 때 볼 것

README에 없는 목표를 만들어 넣지는 않았나요? 어떤 파일을 읽었다고 하는지, 아직 모르는 것은 무엇인지 보세요. AI가 질문하면 모르는 것은 미정으로 남겨도 됩니다.

> **부탁과 강제 차단은 다릅니다**
>
> “수정하지 마세요”는 지시입니다. 권한 설정이나 격리 환경과 같은 보안 장치는 아닙니다. 다음 쪽에서 실제 실행 권한을 따로 배웁니다.

책을 덮고 말해 보세요

AI가 폴더 이름을 말한 것만으로, 파일 내용을 읽었다고 믿어도 될까요?

13. 권한은 AI의 손이 닿는 범위입니다

문서에 “조심해”라고 쓰는 것과, 실행을 제한하는 것은 다릅니다.

규칙 문서

CLAUDE.md에 “중요한 파일을 지우지 말 것”이라고 적으면 AI가 참고할 맥락이 됩니다. 반드시 지킨다는 보장은 아닙니다.

실행 제어

권한 모드·도구 규칙·격리 환경은 무엇을 실행할 수 있는지 제어합니다. 그래도 모든 위험을 없애는 만능 방패는 아닙니다.

구분	뜻
Manual / default	읽기 외의 많은 수정·실행에 확인을 요청합니다. 일부 읽기 전용 명령 등은 예외입니다. 초보 실습에서는 확인하며 진행하는 쪽을 권합니다.
Plan	조사하고 변경 계획을 제안합니다. 소스 파일을 고치지 않도록 설계된 모드지만 “어떤 실행도 절대 안 함”으로 일반화하지 않습니다.
acceptEdits	파일 수정 등을 더 넓게 자동 허용합니다. 초반부터 습관적으로 켜지 않습니다.
auto / bypass	자동 판단 또는 권한 우회 방식입니다. 서로 다르며, 이 교재의 기초 실습에서는 사용하지 않습니다.

수동 검토 중심으로 시작하는 예시 · 설치 버전의 도움말도 확인

```
claude --permission-mode default
```

화면에 뜨는 허용 요청을 읽는 법

어떤 도구인가 → 대상 파일·주소가 무엇인가 → 내 요청에 필요한가 → 한 번 허용인가, 계속 허용인가를 확인합니다. 이해할 수 없는 명령은 거절하고 설명을 요청하세요.

> **버전에 따라 기본값이 다릅니다**
>
> 처음 실행했다고 항상 수동 모드인 것은 아닙니다. 모드 이름을 직접 확인하세요. 규칙 충돌은 현재 공식 문서 기준 deny → ask → allow 순서로 판단합니다.

14. 좋은 요청은 좋은 작업 안내입니다

길게 말하는 능력이 아니라, 빠진 조건을 줄이는 능력입니다.

01 **결과** 누구를 위한 무엇 → 02 **재료** 사실·파일·예시 → 03 **제약** 하지 않을 일 → 04 **확인** 완료 판단 기준

빠진 것이 많은 요청

```
안내문 예쁘게 만들어 줘.
```

더 분명한 요청 · 가상의 행사 예시

```
목표: 처음 오는 방문객을 위한 행사 안내문 초안
독자: 어려운 말을 모르는 사람
자료: 행사명=작은 책 모임, 날짜=미정, 장소=동네 도서관
결과: 제목 1개와 본문 5문장, 쉬운 한국어
방법: 자료에 없는 날짜·요금은 만들지 말고 질문해 주세요.
대화: 빠진 정보는 먼저 두 가지만 물어봐 주세요.
확인: 모든 문장이 위 자료와 일치해야 합니다.
범위: 초안 작성만, 실제 발송은 하지 마세요.
```

구분	뜻
결과물 설명	무엇을, 누구에게, 어떤 형식으로 만들까?
과정 설명	무엇부터 읽고, 어느 지점에서 질문·승인을 받을까?
협력 방식 설명	짧게 답할까? 어려운 말을 풀어 줄까? 내 가정을 비판적으로 봐 줄까?

비밀 주문은 없습니다

“세계 최고 전문가처럼”이라는 말보다 정확한 원자료와 확인 가능한 기준이 도움이 됩니다. 답이 부족하면 요청을 수정하면 됩니다. 처음부터 완벽한 한 문장을 쓸 필요는 없습니다.

책을 덮고 말해 보세요

지금 하는 부탁 하나에 독자·금지 사항·완료 기준을 하나씩 추가해 보세요.

15. 컨텍스트는 지금 펼친 작업대입니다

AI에게 필요한 정보를 주되, 모든 자료를 무조건 넣지는 않습니다.

지금 참고하는 작업대 · 컨텍스트

현재 요청	읽은 파일	이전 대화	도구 결과	규칙·메모

토큰은 무엇인가요?

모델이 텍스트 등을 처리할 때 사용하는 작은 단위입니다. “한 단어 = 한 토큰”처럼 고정되지 않습니다. 컨텍스트 창은 한 번에 참고할 수 있는 범위이고, 사용량 제한은 계정의 이용 규칙입니다. 둘은 같은 개념이 아닙니다.

구분	뜻
필요한 맥락 주기	관련 파일·문서·조건만 선택합니다. 파일이 폴더에 있다는 이유로 전부 읽었다고 가정하지 않습니다.
/context	현재 컨텍스트 사용을 살펴봅니다. 명령 자체가 정보를 줄이거나 답을 정확하게 만드는 것은 아닙니다.
/compact	긴 대화를 요약합니다. 세부 정보가 빠질 수 있으므로 목표·제약·미완료 사항을 다시 확인합니다.
/clear	관련 없는 새 작업으로 바꿀 때 대화 맥락을 정리합니다. 필요한 기록은 먼저 파일에 저장합니다.

> **토큰량과 비용을 섞지 않습니다**
> 자료를 덜 읽으면 처리량을 줄일 여지가 있습니다. 캐시나 더 싼 모델로 비용이 낮아지는 것은 별개의 효과입니다. 서브에이전트를 쓰면 주 대화가 가벼워질 수 있지만, 전체 토큰이 줄어든다고 보장되지 않습니다.

책을 덮고 말해 보세요

AI가 앞서 정한 조건을 놓쳤습니다. 자료를 더 많이 붙이는 것 말고 무엇을 할 수 있나요?

16. 기억은 확인할 수 있는 문서로 남깁니다

"전에 말했으니 알겠지" 대신, 필요한 정보를 짧게 다시 읽게 합니다.

구분	뜻
현재 대화	지금 작업의 이야기. 길어지면 요약되며, 새 작업에 그대로 이어진다고 가정하지 않습니다.
CLAUDE.md	사람이 작성하는 프로젝트 안내·규칙. 새 세션에서도 맥락으로 제공하지만 강제 실행 설정은 아닙니다.
Auto memory	AI가 남기는 학습 메모. 잘못 기록될 수 있으므로 사람이 확인·수정할 수 있어야 합니다.
README.md	사람도 함께 읽는 프로젝트 설명서. 목표와 실행 방법을 적기에 좋습니다.

작은 CLAUDE.md 예시 · 24-28쪽 실습용

```
# 연습 프로젝트
목표: 설치 없이 로컬에서 여는 할 일 목록
범위: index.html과 README.md만 사용
규칙: 쉬운 한국어, 외부 요청·외부 라이브러리 사용 안 함
확인: 27쪽의 수동 검증 항목으로 실제 화면을 확인
주의: 변경 전 계획을 설명하고, 완료 후 변경 파일을 보고
```

짧고 구체적으로

"완벽한 결과를 만들 것"보다 "빈 입력은 항목으로 추가하지 않을 것"이 확인하기 쉽습니다. 처음부터 거대한 규칙집을 쓰지 마세요. 실제 반복 실수가 생겼을 때, 고칠 가치가 있는 규칙을 추가하고 오래된 규칙은 지웁니다.

> **분리했다고 자동으로 토큰이 줄지는 않습니다**
>
> 주제별 .claude/rules/와 경로별 규칙은 심화 도구입니다. 파일을 나누거나 @로 참조해도 함께 읽히면 그만큼 맥락을 차지합니다. 지금은 짧은 프로젝트 안내 하나면 충분합니다.

책을 덮고 말해 보세요

"CLAUDE.md = 영구 기억 = 절대 지킬 규칙"이라고 부르면 왜 부정확한가요?

17. 답을 받은 뒤가 공부의 절반입니다

그냥 다시 하라고 말하지 않고, 어느 기준에서 어긋났는지 알려 줍니다.

01 **설명** 원하는 기준을 전달 → 02 **판단** 결과·방법·협력 점검 → 03 **고치기** 문제를 구체적으로 전달 → 04 **통합** 쓸 것과 버릴 것을 결정

구분	뜻
결과물	사실이 맞나? 독자에게 맞나? 필요한 조건이 빠졌나?
접근 방법	근거를 읽었나? 확인해야 할 것을 건너뛰었나? 적절한 검사를 했나?
대화 방식	질문이 도움이 됐나? 너무 길거나 어려웠나? 내 피드백을 반영했나?

다음 행동을 알기 어려운 피드백

```
별로예요. 더 좋게 해 주세요.
```

유지할 것 + 문제 + 바꿀 방향

```
장소 설명은 유지해 주세요.
날짜는 아직 미정인데 5월 3일로 썼습니다.
날짜를 만들지 말고 "추후 안내"로 바꿔 주세요.
수정한 문장만 보여 주세요.
```

AI의 자기 검토도 보조 도구입니다

AI에게 빠진 점을 찾아 달라고 할 수 있습니다. 하지만 같은 모델이 "검토 완료"라고 말한 것이 독립적인 사실 검증은 아닙니다. 숫자는 원본과, 동작은 실행 결과와, 요구 조건은 체크리스트와 대조합니다.

> **중요한 구분**
> 이 책에서 "과정을 확인한다"는 말은 공개된 작업·근거·검사 결과를 확인한다는 뜻입니다. 모델의 비공개 내부 사고를 볼 수 있다는 의미가 아닙니다.

책을 덮고 말해 보세요

안내문은 예쁘지만 금액이 틀렸습니다. 칭찬·문제·수정 기준을 함께 적어 보세요.

18. 저장과 버전 기록은 다릅니다

파일을 저장하면 현재 상태가 남고, 버전 기록은 바뀐 과정을 남깁니다.

01 **작업 파일** 지금 수정 중인 내용 → 02 **변경 확인** 어디가 바뀌었는지 → 03 **기록할 것 선택** 검토한 파일만 → 04 **커밋** 설명과 함께 상태 기록

구분	뜻
저장	지금 편집한 내용을 파일에 씁니다. 지난 상태가 모두 남는 것은 아닙니다.
백업	복사본을 별도로 보관합니다. 실습 초반에는 폴더 복사도 유용합니다. 동기화만으로 모든 실수 복구가 보장되지는 않습니다.
Git	내 컴퓨터에서 변경 이력을 관리하는 도구입니다. 파일 저장과는 다른 기록 단계가 있습니다.
GitHub	Git 저장소를 온라인에 올리고 협업하는 서비스입니다. Git을 배우기 위해 반드시 공개 업로드할 필요는 없습니다.

이미 Git을 초기화하고 사용자 정보를 설정한 연습 폴더에서의 예시

```
git status
git diff
git add index.html README.md
git diff --staged
git commit -m "로컬 할 일 목록 첫 버전"
```

첫 연습에서는 원격 업로드가 필요 없습니다

Git이 낯설면 먼저 연습 폴더를 “수정 전 복사본”으로 보관하세요. Git을 쓴다면 새 연습 폴더에서 시작하고 사용자 정보 설정을 확인합니다. 아무 폴더에서나 git init을 실행하지 않습니다.

> **주의**
> git add .로 모든 것을 담기 전에 비밀 파일이 없는지 봅니다. 비밀을 커밋했다면 파일만 지워도 과거 기록에 남을 수 있습니다. .git 폴더를 직접 고치거나 지우지 마세요.

19. 읽기 → 계획 → 변경 → 검증 → 기록

빨리 만드는 요령보다, 잘못된 방향으로 오래 가지 않는 습관이 먼저입니다.

> **1. 읽기**
>
> 관련 파일과 요구를 확인합니다. 모르는 것은 질문합니다.

> **2. 계획**
>
> 바꿀 파일·하지 않을 일·확인 방법을 짧게 적습니다.

> **3. 작은 변경**
>
> 한 번에 확인할 수 있는 양만 고칩니다.

> **4. 실제 검증**
>
> 정상 상황뿐 아니라 빈 입력·잘못된 입력도 봅니다.

> **5. 기록**
>
> 바뀐 것·검사한 것·남은 문제를 적고 필요하면 커밋합니다.

책상 정리와 같습니다

방 전체를 한꺼번에 뒤집으면 무엇이 좋아졌는지 알기 어렵습니다. 서랍 하나를 정리하고 물건을 찾을 수 있는지 확인한 뒤 다음 서랍으로 넘어갑니다. AI 작업도 작게 나누면 실패 원인을 찾기 쉽습니다.

> **계획은 길 필요가 없습니다**
>
> 두 줄짜리 수정에 거대한 문서를 만들 필요는 없습니다. 위험이 크고 이해가 부족할수록 계획·확인에 더 신경 쓰세요. 작은 연습은 짧은 계획이면 충분합니다.

책을 덮고 말해 보세요

"완료했습니다"라는 보고에 함께 있어야 할 증거 세 가지는 무엇일까요?

20. 웹 화면은 세 가지를 나눠 봅니다

보이는 모양과 실제 동작을 구분하면, AI에게 고칠 곳을 정확히 말할 수 있습니다.

HTML

무엇이 있나

제목, 입력칸, 버튼, 목록

CSS

어떻게 보이나

글자 크기, 간격, 색, 배치

JavaScript

어떻게 움직이나

버튼을 누르면 항목을 추가

학교 안내판에 비유하면

종이에 제목과 내용을 쓰는 것이 HTML, 보기 좋게 크기와 간격을 정하는 것이 CSS, 버튼을 눌러 내용을 바꾸게 하는 것이 JavaScript입니다. 서로 역할이 다르지만 한 파일에 함께 넣을 수도 있습니다.

구분	뜻
“글씨를 크게 해 주세요”	주로 표시·스타일 문제입니다.
“버튼을 눌러도 추가가 안 돼요”	주로 동작 문제입니다. 실제 누른 절차와 오류를 기록합니다.
“새로 열면 항목이 없어져요”	저장 방식의 문제일 수 있습니다. 원래 요구에 저장 기능이 있는지부터 봅니다.

코드 언어를 모두 외울 필요는 없습니다

이름, 값, 조건, 반복 같은 개념은 천천히 배워도 됩니다. 다만 AI가 만든 코드가 어느 파일에 들어 있고, 어떻게 실행하며, 어떻게 확인하는지는 알아야 합니다.

> **프레임워크는 나중에**
>
> React·Next.js 같은 도구는 프로젝트를 구성하는 데 도움이 될 수 있습니다. 하지만 제목 하나를 바꾸는 연습에 처음부터 복잡한 도구를 모두 도입할 필요는 없습니다.

책을 덮고 말해 보세요

화면이 예쁘게 나온 것과, 기능이 맞게 작동하는 것을 어떻게 따로 확인할까요?

21. 내 컴퓨터와 인터넷을 구분합니다

파일을 열었다고 모두가 볼 수 있는 웹사이트가 된 것은 아닙니다.

01
브라우저
사용자에게 보이는 창

→

02
서버
요청을 받아 처리

→

03
데이터베이스
정보를 저장·조회

구분	뜻
로컬 파일	내 컴퓨터의 파일을 브라우저로 엽니다. 다음 실습은 여기까지만 합니다.
로컬 서버	내 컴퓨터에서 서버 프로그램을 실행합니다. localhost는 보통 내 컴퓨터를 가리킵니다. 공개 배포와 다릅니다.
배포	다른 사용자가 접속할 수 있는 환경에 올립니다. 접근 권한·비용·보안·운영 책임이 추가됩니다.
API	프로그램끼리 정해진 방식으로 요청·응답하는 접점입니다. 인증 키는 출입 열쇠처럼 보호합니다.
데이터베이스	정보를 구조적으로 보관합니다. 화면에서 안 보인다고 지워진 것도, 화면에서 보인다고 접근 통제가 안전한 것도 아닙니다.

권한은 화면보다 안쪽에서도 확인합니다

버튼을 숨겨도 서버나 데이터가 보호되는 것은 아닙니다. 실제 서비스에서는 로그인, 데이터 접근 규칙, 입력 검증, 비밀 관리가 필요합니다. 그래서 Notion API·Supabase·인증·배포 실습을 이 책의 첫 프로젝트에서 제외했습니다.

> **기초 실습 범위**
> 로그인·결제·고객정보·외부 API를 쓰지 않습니다. 작은 결과를 읽고 수정하고 확인하는 한 바퀴부터 익히세요. 인터넷 서비스 만들기는 이 한 바퀴를 설명할 수 있게 된 뒤의 선택 학습입니다.

책을 덮고 말해 보세요

“내 컴퓨터에서 열림”과 “다른 사람에게 공개됨” 사이에 추가로 필요한 확인은 무엇인가요?

22. 오류는 정답으로 가는 관찰 자료입니다

“안 돼요”보다, 무엇을 기대했고 실제로 무엇이 일어났는지가 중요합니다.

01 **다시 해 보기** 같은 조건·순서 → 02 **차이 기록** 기대와 실제 → 03 **가설 하나** 의심 원인 좁히기 → 04 **작게 수정** 한 가지 바꾸고 재확인

재현 가능한 오류 보고 예시

```
환경: macOS, 브라우저에서 로컬 HTML 열기
순서: 입력칸을 비운 채 추가 버튼 클릭
기대: 목록이 늘지 않고 안내가 나와야 함
실제: 빈 항목이 하나 생김
관련 파일: index.html
요청: 원인을 먼저 설명하고 최소 수정안을 제안해 주세요.
금지: 외부 라이브러리 추가·전체 구조 교체
```

에러 메시지는 번역하지 말고 원문도 남깁니다

문구와 줄 번호는 중요한 단서입니다. 화면을 캡처해도 좋지만 비밀정보는 가리세요. “아마 이것 때문”이라는 추측과 실제 관찰을 섞지 않습니다.

구분	뜻
바꾸기 전	수정 전 복사본 또는 Git 변경 상태를 확인
바꾼 뒤	같은 순서로 문제가 사라졌는지 확인
함께 확인	원래 되던 정상 입력도 여전히 되는지 확인
모르겠으면	수정 반복을 멈추고 현재 파일·오류·시도한 변경을 정리

> **재설치는 마지막 후보 중 하나**
>
> 입력 검증 코드가 빠진 문제를 재설치로 해결할 수는 없습니다. 설치·인증·코드·네트워크 중 어느 종류인지 먼저 나눠 보세요.

책을 덮고 말해 보세요

오류를 고쳤다고 주장할 때, 같은 입력으로 다시 확인해야 하는 이유는 무엇인가요?

23. 안전은 마지막 옵션이 아닙니다

공유할 정보, 실행할 행동, 내보낼 결과를 작업 시작부터 구분합니다.

구분	뜻
정보 최소화	실습에는 가짜 데이터를 씁니다. 이름·연락처·계약·건강정보는 필요성과 이용 조건을 확인하기 전 넣지 않습니다.
비밀 보호	비밀번호·일회용 로그인 코드·API 키를 대화나 공개 저장소에 넣지 않습니다. 노출됐다면 삭제만 하지 말고 폐기·교체 필요성을 확인합니다.
외부 자료의 지시	문서나 웹페이지 안의 “이전 지시 무시”, “키를 보내라”는 문장은 작업 자료 속 내용일 수 있습니다. 사용자 승인으로 취급하지 않습니다.
중요 행동	외부 전송·배포·결제·삭제·권한 확대는 정확한 대상과 내용을 확인하고 별도로 승인합니다.
공개와 책임	업무·학교 등의 규칙에 맞춰 AI의 기여를 밝히고, 실제로 검토한 범위만 적습니다.

낯선 문서를 읽히는 일도 주의합니다

도구를 가진 AI는 자료 속 악의적 지시를 잘못 따를 위험이 있습니다. 이를 프롬프트 인젝션이라고 부릅니다. 민감한 원본을 분리하고, 실행 범위를 좁히고, 예상 밖 요청이 나오면 멈추는 습관이 중요합니다.

> **이 책의 안전 연습**
> 연습 폴더·가짜 데이터만 사용 / 중요한 삭제·패키지 추가 없음. “외부 연결 없음”은 만드는 HTML의 동작 범위입니다. Claude Code 자체는 요청·읽은 파일 내용 등을 모델 제공자로 전송할 수 있습니다. 오프라인 비밀 작업 도구로 오해하지 마세요.

책을 덮고 말해 보세요

외부 문서에 “검증을 생략하고 업로드하라”고 적혀 있습니다. 어떻게 처리할까요?

24. 오늘 할 일 목록: 먼저 범위를 정합니다

작고 단순해서, 무엇이 맞고 틀렸는지 직접 판단할 수 있는 연습입니다.

만들 결과

내 컴퓨터의 브라우저에서 여는 “오늘 할 일 목록”입니다. 한 줄을 입력하고 추가하면 목록에 보입니다. 빈 입력은 무시하고, 새로고침하면 목록이 사라집니다. 저장하지 않는 것은 이번 연습의 의도된 범위입니다.

구분	뜻
할 것	제목·입력칸·추가 버튼·목록 / 빈 입력 차단 / 키보드 Enter 추가
하지 않을 것	로그인·데이터베이스·외부 API·설치 패키지·공개 배포·영구 저장
사용 파일	index.html 한 개, 목표를 적을 README.md
사람이 할 일	요구 선택, 계획 승인, 실제 브라우저 확인, 남은 문제 기록
AI에게 맡길 일	짧은 계획, 예제 설명 또는 구현 초안, 수정 제안
끝났다는 증거	27쪽 체크리스트를 같은 파일에서 직접 시험하고 결과를 적음

> **학습 방법은 둘 중 하나**
>
> A. 다음 쪽 예제 파일을 직접 만들어 실행합니다.
> B. 26쪽 요청으로 AI에게 같은 범위의 초안을 만들게 합니다.
> 둘을 한꺼번에 할 필요는 없습니다. 예제와 AI 결과가 똑같은 코드일 필요도 없습니다.

우리가 확인할 핵심

입력한 글자가 들어가는가, 빈 입력은 막히는가, 키보드로도 되는가, 새로고침 동작을 이해하는가입니다. 화려한 디자인과 자동화는 평가 항목이 아닙니다.

책을 덮고 말해 보세요

“새로고침 후 사라짐”이 이번에는 버그가 아닌 이유를 설명해 보세요.

25. 직접 열 수 있는 최소 예제

아래 코드를 index.html로 저장한 뒤 브라우저에서 엽니다.

자체 작성 참고 코드 · 외부 라이브러리·네트워크 요청 없음

```
<!doctype html>
<html lang="ko">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>오늘 할 일</title>
<style>
body{max-width:38rem;margin:2rem auto;padding:1rem;
  font:18px/1.7 system-ui,sans-serif;color:#222}
input,button{font:inherit;padding:.4rem}
li{overflow-wrap:anywhere}
</style>
<h1>오늘 할 일</h1>
<p>새로고침하면 목록이 사라지는 연습 문서입니다.</p>
<form id="form">
  <label for="task">할 일</label>
  <input id="task" maxlength="80" required>
  <button>추가</button>
</form>
<p id="message" role="status"></p>
<ul id="list"></ul>
<script>
const form = document.querySelector('#form');
const input = document.querySelector('#task');
const list = document.querySelector('#list');
const message = document.querySelector('#message');
form.addEventListener('submit', event => {
  event.preventDefault();
  const text = input.value.trim();
  if (!text) { message.textContent='할 일을 적어 주세요.'; return; }
  const item = document.createElement('li');
  item.textContent = text;
  list.append(item);
  input.value = '';
  message.textContent = '추가했습니다.';
  input.focus();
});
</script>
</html>
```

문서 편집기의 서식 있는 파일이 아닌 **일반 텍스트**로 저장합니다. 파일 이름이 index.html.txt가 되지 않았는지 확인하세요. 빈 입력에는 브라우저의 기본 안내가 표시될 수 있고, 공백만 입력하면 코드의 안내가 표시됩니다.

26. AI에게 맡겨도 범위는 내가 정합니다

계획을 보고 승인한 다음, 한 파일만 만들게 합니다.

먼저 보낼 요청

```
연습 폴더에 로컬 할 일 목록을 만들려고 합니다.
24쪽과 같은 조건으로 먼저 계획만 제안해 주세요.
- index.html 한 개, 외부 연결과 라이브러리 없음
- 할 일 입력, 버튼 또는 Enter로 추가
- 빈 문자열과 공백만 있는 입력은 추가하지 않음
- 입력한 글자는 HTML이 아니라 일반 글자로 표시
- 새로고침하면 목록은 사라짐, 이 점을 화면에 안내
- 입력칸에 label, 결과 안내에 접근 가능한 상태 표시
계획에 바꿀 파일, 금지 사항, 수동 확인 항목을 넣어 주세요.
아직 파일 생성·수정은 하지 마세요.
```

- [ ] 폴더와 변경 파일이 맞는지 확인했습니다.
- [ ] 로그인·설치·외부 연결이 추가되지 않았습니다.
- [ ] 빈 입력·Enter·새로고침 확인이 계획에 있습니다.

계획이 적절할 때만 승인하는 예시

```
이 계획대로 index.html만 만들어 주세요.
도구를 통한 외부 서비스 추가 전송·패키지 설치·배포·다른 파일 수정은 하지 마세요.
완료 후 실제 변경 파일과 아직 검사하지 못한 항목을 구분해 주세요.
```

권한 요청이 나오면

"계획을 승인했다"가 모든 후속 명령을 승인한 것은 아닙니다. 지금 허용 요청의 대상과 동작을 읽으세요. 계획에 없는 설치나 네트워크 요청이라면 멈추고 이유를 확인합니다.

> **완료 보고의 좋은 형태**
>
> "파일 작성 완료 / 수동 화면 검증은 미실시"처럼 사실을 나눠 적어야 합니다. 코드를 썼다는 사실만으로 모든 테스트가 통과한 것은 아닙니다.

27. 체크리스트로 직접 확인합니다

AI에게 검사를 시켜도 좋지만, 처음 한 바퀴는 내가 결과를 봅니다.

번호	내가 할 행동	기대하는 결과
01	제목·입력칸·버튼을 봅니다.	할 일을 적는 곳이 무엇인지 알 수 있음
02	“물 마시기”를 넣고 추가합니다.	그 글자 항목이 정확히 하나 추가됨
03	“책 읽기”를 넣고 Enter를 누릅니다.	두 번째 항목이 추가되고 페이지가 이동하지 않음
04	입력을 비우고 추가합니다.	빈 항목이 생기지 않음
05	공백만 넣고 추가합니다.	빈 항목이 생기지 않고 안내를 볼 수 있음
06	연습를 넣습니다.	꺾쇠와 태그 이름까지 문자열 그대로 보임
07	긴 글자를 넣습니다.	화면이 크게 깨지지 않음, 입력 길이 제한 확인
08	새로고침합니다.	목록이 사라짐. 화면의 설명과 일치함
09	창을 좁히고 키보드 Tab으로 이동합니다.	내용을 읽고 입력·버튼에 접근할 수 있음

> **결과는 실제대로**
> 통과 / 실패 / 미검사 중 하나로 표시하세요. 화면이 “깔끔하다”는 평가는 기능 검증을 대체하지 않습니다. 한 항목이 실패하면 30쪽 오류 기록지로 옮깁니다.

내가 확인한 파일·날짜·브라우저

실패 또는 미검사 항목

28. 한 가지 바꾸고 다시 확인합니다

실패를 없애는 것뿐 아니라, 왜 바꾸는지 설명하는 연습입니다.

두 번째 연습: 삭제 기능

첫 버전 검증이 끝나면, 항목 옆에 삭제 버튼을 하나 추가해 봅니다. 전체 초기화·저장·로그인까지 동시에 추가하지 않습니다. 수정 전 파일을 복사하거나 Git으로 기록해 두세요.

작은 기능 확장 요청

```
검증한 첫 버전에 항목별 삭제만 추가하려고 합니다.
먼저 변경 계획과 확인 항목을 알려 주세요.
삭제 버튼은 어떤 항목을 지우는지 알 수 있어야 합니다.
선택한 항목 하나만 지우고 다른 항목은 유지해야 합니다.
빈 입력 차단·Enter 추가는 그대로 유지해 주세요.
외부 도구·패키지·영구 저장은 추가하지 마세요.
```

- [ ] 첫 항목을 지워도 둘째 항목은 남습니다.
- [ ] 같은 글자 항목이 두 개여도 누른 항목만 사라집니다.
- [ ] 마지막 항목을 지워도 오류가 없습니다.
- [ ] 원래의 추가·빈 입력·Enter 동작도 그대로 됩니다.

잘못되면 되돌립니다

AI에게 “다 고쳐”라고 반복하기 전에 변경 차이를 봅니다. 복사본과 비교하거나 Git diff로 확인하세요. Git 복구 명령은 범위를 이해한 뒤 사용합니다. 중요한 미저장 변경이 있는 상태에서 강제 초기화하지 않습니다.

> **짧은 회고**
>
> 무엇을 바꿨나: _____
> 어떤 증거로 확인했나: _____
> 같은 실수를 줄이려면 무엇을 바꿀까: _____

책을 덮고 말해 보세요

기능 하나를 추가했는데, 이전 기능도 다시 확인해야 하는 이유는 무엇인가요?

29. 요청을 준비하는 한 장

실제 파일을 바꾸기 전에 이 칸부터 채워 보세요.

만들 결과 한 문장

누가 어떤 상황에서 쓸 것인가요?

원본 자료와 출처

사실로 써도 되는 내용, 모르는 내용을 구분하세요.

AI에게 맡길 부분 / 내가 결정할 부분

변경 가능한 파일·행동 / 금지할 행동

외부 전송·비용·삭제·설치·권한 확대를 특히 확인합니다.

끝났다고 판단할 기준

눈으로 보거나 실행해서 확인할 수 있는 말로 적습니다.

멈추고 질문해야 하는 경우

30. 오류와 검증을 남기는 한 장

사실을 남겨야 다음 대화에서도 같은 지점에서 이어갈 수 있습니다.

환경·파일·관찰 날짜

운영체제 / 셸 또는 브라우저 / 파일 이름·경로

문제를 재현하는 순서

다른 사람이 같은 현상을 볼 수 있을 만큼 구체적으로

기대 결과 / 실제 결과

오류 원문·증거 위치

비밀번호·키·개인정보는 가리고 공유합니다.

바꾼 것 / 다시 검사한 것

남은 문제·미검사·다음 한 가지

31. 혼자 설명할 수 있으면 통과입니다

정답 문장을 외우기보다, 직접 겪은 예로 말해 보세요.

- ☐ 모델·도구·터미널을 구분할 수 있습니다.
- ☐ 현재 폴더와 바꿀 파일을 확인할 수 있습니다.
- ☐ 원자료·목표·금지 사항·완료 기준을 넣어 요청할 수 있습니다.
- ☐ CLAUDE.md의 지시와 실행 권한의 차이를 설명할 수 있습니다.
- ☐ AI의 완료 보고와 실제 검증 결과를 구분할 수 있습니다.
- ☐ 오류를 재현 순서와 기대·실제로 나눠 적을 수 있습니다.
- ☐ 작은 변경을 기록하고 필요한 경우 되돌릴 준비를 할 수 있습니다.

되짚어 볼 질문

① 컨텍스트가 많으면 항상 좋을까요?
② 새로고침 후 목록이 사라지는 것은 항상 오류일까요?
③ AI가 만든 안내문을 실제로 공유하기 전에 무엇을 봐야 할까요?
④ 더 싼 모델을 썼다면 토큰도 줄었다고 말해도 될까요?

> **생각의 방향**
>
> ① 필요한 정보가 중요하며, 무관한 내용은 방해가 될 수 있습니다.
> ② 요구한 저장 방식에 따라 다릅니다. 이번 실습에서는 의도된 동작입니다.
> ③ 사실·날짜·금액·독자 적합성·개인정보·공개 규칙과 실제 검토 범위를 봅니다.
> ④ 아닙니다. 처리량과 단가는 구분해야 합니다.

다음 단계로 가도 될까요?

위 항목 중 설명할 수 없는 것이 있으면 해당 단원을 다시 읽으세요. 정해진 점수나 제한 시간은 없습니다. 설명할 수 있는 것 하나가 늘어나는 것이 학습의 진도입니다.

32. 고급 기능은 문제가 생긴 뒤 고릅니다

도구를 먼저 모으지 말고, 어떤 불편을 해결할지부터 정합니다.

실제 문제	후보 도구	도입 전 질문
같은 절차를 자주 반복	스킬·사용자 명령	짧은 수동 절차가 실제로 반복되는가?
외부 서비스 정보 필요	MCP·API 연결	접근 범위·전송 정보·인증·신뢰성을 설명할 수 있는가?
작업마다 검사 누락	훅·자동 테스트	검사 자체가 올바른가? 실패하면 실제로 멈추는가?
독립 조사로 주 대화 비대	서브에이전트	명확하게 분리 가능한 일인가? 반환 내용·비용은?
작업자 간 조율 필요	에이전트 팀	단일 작업자보다 이점이 있는가? 충돌·비용을 통제할 수 있는가?
대규모 앱·사용자 데이터	프레임워크·DB·배포	인증·데이터 접근·장애·비용을 검증할 준비가 됐는가?

하네스라는 말의 쉬운 뜻

AI가 일하는 주변 환경입니다. 규칙, 도구, 권한, 검사, 기록이 함께 포함됩니다. 새로운 마법 기술을 뜻하지 않습니다. 지금까지 만든 연습 폴더와 규칙·체크리스트도 그 출발점입니다.

> **최신 강의를 기초에 반영한 방식**
>
> 162번의 간결한 규칙은 16쪽에, 165번의 환경·검증은 13·19·23쪽에, 167번의 측정 우선은 15쪽에 반영했습니다. 159·160번 팀 운영은 필요성과 비용을 먼저 묻는 선택 학습으로 남겼습니다.

플러그인·훅이 있다고 무조건 안전해지거나 비용이 줄지 않습니다. 이 교재는 도구 설치나 무인 자동 운영을 승인·실행한 기록이 아닙니다.

33. 그대로 외우면 안 되는 설명

원본·강사 교안의 오래되거나 강한 표현을 현재 문서와 구분했습니다.

주의할 표현	이 책의 처리	근거·연결
Node.js·npm부터 필수 설치	웹 앱 개발에는 필요할 수 있지만, 현재 공식 네이티브 설치 경로와 분리합니다. 필요한 프로젝트에서 추가합니다.	10쪽 · D1
CLAUDE.md는 영구 기억	세션에 제공되는 문서 맥락입니다. 무한 기억·절대 준수·보안 강제 장치가 아닙니다.	16쪽 · D6
권한 우선순위 deny → allow → ask	현재 공식 문서의 규칙 평가는 deny → ask → allow입니다. 오래된 예시를 그대로 복사하지 않습니다.	13쪽 · D4
Shift+Tab 몇 번이면 항상 특정 모드	버전·설정·사용 화면에 따라 다를 수 있습니다. 현재 모드 표시를 봅니다. 처음 모드도 가정하지 않습니다.	13쪽 · D3
think hard·ultrathink의 고정 단계	현재 모델·버전의 사고 설정과 별개로 확인해야 합니다. 이 책은 주문 같은 키워드 암기를 가르치지 않습니다.	D7 모델 설정
화면이 예쁘면 검증 완료	외형과 기능·데이터·권한 검사는 다릅니다. 실제 사용자 동작을 확인합니다.	17·27쪽
규칙 분리·캐시·팀으로 토큰 절약 보장	불필요한 읽기 감소, 캐시 가격 효과, 모델 단가 효과, 병렬 작업 총량을 따로 측정해야 합니다.	15쪽 · 최신167
훅·권한은 우회 불가	적용 범위·도구·실패 처리·격리 환경을 따로 검증해야 합니다. 만능 보안 보장으로 쓰지 않습니다.	23·32쪽 · D4

> **화면·명령·요금은 바뀝니다**
>
> 현재 문서와 내 설치 버전이 다를 수 있습니다. 버전 번호·확인 날짜를 남기고, 화면이 다르면 명령을 무작정 반복하지 마세요.

34. 최신 추가 수업을 어떻게 반영했나

첨부 자료에서 확인한 핵심을, 초보자에게 필요한 만큼만 본문에 연결했습니다.

수업 번호·제목 날짜 코드	강사 첨부에서 확인한 내용	이번 편집의 선택
159·160 / 2026.02.27·03.05	팀과 서브에이전트 비교, 계획 승인, 비용·검사 설계.	32쪽: 혼자 할 수 있으면 먼저 단순하게. 팀 도입·모델 믹싱은 의무 아님.
162 / 2026.03.17	CLAUDE.md를 짧고 구체적으로. 검증 방법·도메인 용어, 반복 실수에서 개선.	16쪽: 작은 규칙 예시. 문서와 강제 설정 구분.
165 / 2026.04.11	규칙·권한·훅·검사 도구·생성과 평가 분리·환경 개선.	13·19·23·32쪽: 작동 환경을 이해하고 결과를 확인.
167 / 2026.05.01	먼저 측정, 문서 정리, 필요한 부분 읽기, 반복 실수 방지, 재측정.	15·32쪽: 도구 설치보다 측정·필요성 판단.

무엇까지 읽었나요?

159·160·162의 첨부 HTML 전체, 165의 첨부 HTML 5개, 167의 첨부 HTML 전체를 텍스트로 확인했습니다. 스크립트는 각 수업의 처음 약 1분대만 확인했습니다. 전체 영상을 시청한 것은 아닙니다.

반영하지 않은 강한 주장

“비용 절반”, “품질 몇 배”, “같은 실수 반복 없음” 같은 표현은 모든 환경에서 검증된 효과가 아닙니다. 플러그인 이름은 수업 당시 정보이며 이번 기초 실습의 필수 도구로 지정하지 않았습니다.

> **날짜 해석**
>
> 위 날짜는 제목의 UP_YYMMDD 코드입니다. 최초 업로드·촬영일을 독립 확인한 값이 아닙니다. 강좌 소개의 마지막 업데이트 표시는 2026.08.18, 이번 열람일은 2026.10.21입니다.

35. 강좌 연결표 1 / 3

새 교재 순서로 공부하고, 더 자세한 예시가 필요할 때 원 강의를 찾아갑니다.

인프런 섹션	이 책 연결·우선순위	재구성 방향
1. 시작하기 전에	4-7쪽 우선	모델과 도구, 목표와 학습 태도. 오리엔테이션·교안 위치 확인.
2. 터미널 익숙해지기	8-9쪽 우선	위치·목록·이동. 명령 암기보다 셸과 경로 구분.
3. 개발환경 및 설치	10-11쪽 우선	필수·선택 도구 분리. 현재 공식 설치와 대조.
4. 맛보기 및 초기화	12·16쪽 우선	첫 읽기 작업, /init과 프로젝트 안내의 의미.
5. IDE 통합	4·10쪽 필요할 때	편집기와 모델 구분. 특정 편집기 강제하지 않음.
6. 클로드 코드 권한	13·23쪽 우선	모드·승인·범위. 사고 키워드·UI 세부는 버전 확인.
7. 슬래시 명령·단축키	15쪽 일부 우선	컨텍스트 확인·정리부터. 꾸미기 기능은 후순위.
8. Git과 GitHub	18쪽 우선	저장·변경 비교·커밋·원격 업로드를 분리.
9. 설정·메모리 관리	13·16쪽 우선	문서 지시와 실행 설정 구분. 상세 스키마는 심화.
10. MCP 활용	21·32쪽 나중	연결할 이유·정보 전송·권한을 설명한 뒤.

> **제목을 따라 최신 내용을 찾으세요**
>
> 섹션 번호는 2026.10.21에 확인한 목차 기준입니다. 안내·미션·퀴즈가 섞여 있어 화면마다 개수가 다를 수 있습니다. 이 표는 수강 진도를 대신 표시하거나 모든 영상 확인을 증명하지 않습니다.

36. 강좌 연결표 2 / 3

새 교재 순서로 공부하고, 더 자세한 예시가 필요할 때 원 강의를 찾아갑니다.

인프런 섹션	이 책 연결·우선순위	재구성 방향
11. 기술스택·워크플로우	19-21쪽 개념 우선	도구 목록보다 읽기·계획·검증의 흐름.
12. Starter Kit: AI 활용	24-28쪽 축소 연습	원 강좌의 대형 스타터킷 대신 한 파일 실습.
13. Starter Kit: 공식문서	17·20·22쪽 기초 후	문서 읽기·오류 관찰부터. Next.js 구성은 별도 수강.
14. 커스텀 커맨드	32쪽 나중	반복이 확인되면 스킬·명령 검토. 통합 변경 주의.
15. 서브에이전트	15·32쪽 나중	분리 가능한 조사와 컨텍스트·총량의 차이.
16. 훅	13·32쪽 나중	자동 검사가 필요한 실제 반복 실패부터.
17. MCP Server 설치	21·23·32쪽 나중	도구 신뢰·범위·연결 목적 확인 뒤 설치.
18. Notion 견적서 시작	7·14·24쪽 기초 후	목표·독자·범위·MVP 사고만 먼저 적용.
19. 작업 관리 도구	7·19쪽 도구 나중	작업 나누기를 먼저. 전용 관리도구는 필수 아님.
20. ROADMAP 만들기	19·29쪽 기초 후	다음 작은 일과 완료 기준부터 문서화.

제목을 따라 최신 내용을 찾으세요

섹션 번호는 2026.10.21에 확인한 목차 기준입니다. 안내·미션·퀴즈가 섞여 있어 화면마다 개수가 다를 수 있습니다. 이 표는 수강 진도를 대신 표시하거나 모든 영상 확인을 증명하지 않습니다.

37. 강좌 연결표 3 / 3

새 교재 순서로 공부하고, 더 자세한 예시가 필요할 때 원 강의를 찾아갑니다.

인프런 섹션	이 책 연결·우선순위	재구성 방향
21. Shrimp 도입	32쪽 나중	수동 작업 관리의 실제 불편을 확인한 뒤.
22. MVP 개발·배포	21·23쪽 나중	API 키·외부 연결·배포 책임이 있는 별도 실습.
23. 관리자 기능 확장	28쪽 기초 후	작은 변경·회귀 확인을 먼저. 관리자 권한은 별도 학습.
24. Supabase 시작	21·23쪽 나중	DB·인증·RLS 등 데이터 보호 개념 후.
25. 이벤트 앱 시작	7·24·29쪽 기초 후	문제·범위 정의부터. 앱 요구 전체는 원 강의에서.
26. 이벤트 앱 구현	19·22·27쪽 나중	구현·사용자 흐름 테스트 습관만 기초에서 연습.
27. 스킬·플러그인	32쪽 나중	반복 절차가 검증된 뒤 재사용·공유.
28. 주제별 심화	15·32쪽 나중	문제 원인을 측정한 뒤 최적화.
29. Agent Teams	32·34쪽 나중	상호 조율이 필요한 경우만. 첨부 159·160 확인.
30. 실전 활용 심화	16·32-34쪽 선별 반영	162·165·167 첨부 반영. PPT·카드뉴스 등은 후순위.

제목을 따라 최신 내용을 찾으세요

섹션 번호는 2026.10.21에 확인한 목차 기준입니다. 안내·미션·퀴즈가 섞여 있어 화면마다 개수가 다를 수 있습니다. 이 표는 수강 진도를 대신 표시하거나 모든 영상 확인을 증명하지 않습니다.

38. 처음 만나는 말, 한 문장 풀이

모르는 말이 나오면 문장을 멈추고 여기에서 뜻을 확인하세요.

용어	이 책의 쉬운 풀이
모델	주어진 정보를 바탕으로 응답을 생성하는 AI의 핵심 부분.
도구	파일 읽기·수정·명령 실행처럼 실제 행동을 하는 기능.
프롬프트	AI에게 보내는 요청이나 안내.
컨텍스트	지금 응답을 만들 때 참고하는 정보 묶음.
토큰	모델이 정보를 처리할 때 쓰는 단위. 단어나 글자와 고정 일치하지 않음.
세션	이어지는 하나의 대화·작업 단위. 재개·요약 방식은 제품에 따라 다름.
터미널 / 셸	명령을 주고받는 창 / 명령을 해석하는 프로그램.
경로	파일이나 폴더의 위치를 나타내는 주소.
CLI / IDE	명령줄 인터페이스 / 코드 편집 등 개발 기능을 모은 환경.
의존성·패키지	프로젝트가 이용하는 다른 코드·라이브러리 묶음.
Git / 커밋	변경 이력을 관리하는 도구 / 검토한 상태를 기록하는 단위.
API / 키	프로그램 간 요청 접점 / 접근을 인증하는 비밀 값일 수 있는 정보.
MCP	AI 애플리케이션과 외부 도구·데이터를 연결하는 표준 방식.
하네스	모델이 도구·맥락·권한·검사를 이용하며 일하게 하는 주변 구조.
테스트 / 회귀 확인	기대와 실제를 비교 / 변경 후 원래 기능이 깨지지 않았는지 확인.
로컬 / 배포	내 환경에서 실행 / 다른 사용자가 접근할 운영 환경 등에 올림.
스킬 / 훅	재사용 절차·지식 묶음 / 특정 사건 때 실행되는 검사·자동화 장치.

정밀한 제품별 정의는 공식 문서가 우선합니다. 이 풀이는 초보 학습을 위한 축약 설명입니다.

39. 강의와 공식 교육 출처

요약은 원문 전체의 번역이 아닙니다. 더 깊게 배울 때 원 강의를 함께 보세요.

I0 · 인프런 강좌 소개·전체 최신 목차
https://www.inflearn.com/course/클로드-코드-완벽-마스터-ai-개발

I1 · 강사 Notion 교안 루트
https://gymcoding.notion.site/Claude-Code-with-AI-24c6a10d310b8077b064eca98557a416

F0 · AI Fluency: Framework and foundations
https://academy.claude.com/courses/ai-fluency-framework-foundations

F1 · The 4D Framework
https://academy.claude.com/courses/ai-fluency-framework-foundations/the-4d-framework

F2 · Generative AI fundamentals
https://academy.claude.com/courses/ai-fluency-framework-foundations/generative-ai-fundamentals

F3 · Capabilities & limitations
https://academy.claude.com/courses/ai-fluency-framework-foundations/capabilities-limitations

F4 · A closer look at Delegation
https://academy.claude.com/courses/ai-fluency-framework-foundations/a-closer-look-at-delegation

F5 · Project planning and Delegation
https://academy.claude.com/courses/ai-fluency-framework-foundations/project-planning-and-delegation

F6 · A closer look at Description
https://academy.claude.com/courses/ai-fluency-framework-foundations/a-closer-look-at-description

F7 · A closer look at Discernment
https://academy.claude.com/courses/ai-fluency-framework-foundations/a-closer-look-at-discernment

F8 · The Description-Discernment loop
https://academy.claude.com/courses/ai-fluency-framework-foundations/the-description-discernment-loop

F9 · A closer look at Diligence
https://academy.claude.com/courses/ai-fluency-framework-foundations/a-closer-look-at-diligence

> **인프런 첨부 자료의 열람 경로**
>
> 수업 159 unitId=419533 / 160=419540 / 162=426666 / 165=433551 / 167=441056. 로그인 후 강좌 커리큘럼에서 해당 번호·제목으로 찾을 수 있습니다. 수업 제목의 날짜 코드는 34쪽에 구분했습니다.

40. 최신 문서와 검증 범위

버전이 바뀌는 설명은 현재 공식 문서와 내 환경을 다시 대조합니다.

D1 · Quickstart
https://code.claude.com/docs/en/quickstart

D2 · How Claude Code works
https://code.claude.com/docs/en/how-claude-code-works

D3 · Choose a permission mode
https://code.claude.com/docs/en/permission-modes

D4 · Configure permissions
https://code.claude.com/docs/en/permissions

D5 · Context window
https://code.claude.com/docs/en/context-window

D6 · How Claude remembers your project
https://code.claude.com/docs/en/memory

D7 · Model configuration
https://code.claude.com/docs/en/model-config

공식 과정 반영의 범위

AI Fluency 관련 9레슨의 Summary·Key takeaways·Exercises를 확인했습니다. 공식 영상 전문은 확보하지 않았습니다. Claude Code in action 소개도 확인했으나, 이 과정의 장시간 자동 운영을 기초 학습의 필수 경로로 삼지 않았습니다.

제작자의 AI 사용·검토 진술

AI가 내용 재구성·한국어 풀이·예제·도식을 작성하고, 수강 교안·최신 첨부와 공식 문서를 대조했습니다. 설명·비유·학습 경로는 편집 제안이며 학습 효과를 사용자 시험으로 검증한 결과는 아닙니다. 실제 이용·공유 전 필요한 검토와 책임은 사용 맥락에 맞게 판단해야 합니다.

예제·인쇄 검증

참고 예제를 브라우저에서 실행해 정상 추가·Enter·빈 입력·공백·태그 문자열·새로고침·긴 입력·키보드 이동을 확인했습니다. 380px 폭에서도 내용 넘침을 점검했습니다. 본문·바닥글 겹침 검사와 A4 PDF 40쪽의 쪽수·한글 추출을 확인하고 대표 페이지를 시각 점검했습니다. 이 결과는 참고 예제에 대한 제한된 확인이며, AI가 나중에 만든 다른 코드의 동작이나 일반 보안성을 보장하지 않습니다.

AI Fluency 원자료에는 ©2025 Rick Dakan, Joseph Feller, Anthropic · CC BY-NC-SA 4.0 표시가 있습니다. 본 교재의 해당 교육 내용에 기반한 풀이·개작 부분은 출처를 표시하고 비영리·동일조건변경허락 조건을 유지합니다. 별도 강사 자료의 권리까지 이 라이선스로 바뀌지는 않습니다. 개인 학습용으로 사용하고, 판매·재배포 전 권리를 별도 확인하세요.